VEB DASTURLAR XAVFSIZLIGI

A8. API xavfsizligi va zamonaviy autentifikatsiya mexanizmlari

1-VARIANT — O‘quv buyurtmalar API xizmatini himoyalash

Mashg‘ulot maqsadi. JWT orqali kelgan foydalanuvchi identifikatorini tekshirish, buyurtma egasi va administrator huquqlarini alohida nazorat qilish hamda autentifikatsiyadan o‘tgan mijoz uchun so‘rovlar tezligini cheklash. Har bir himoya natijasi HTTP javobi va qonuniy nazorat so‘rovi bilan asoslanadi.

Kutiladigan natija. Talaba imzo va claim tekshiruvini oddiy dekodlashdan, autentifikatsiyani obyekt hamda funksiya darajasidagi avtorizatsiyadan ajratadi. Uchta zaif funksiyani tuzatadi va aynan avvalgi endpointlarda qayta sinaydi. API kaliti, GraphQL va API Gateway vazifalarini misol bilan izohlaydi.

Ko‘rsatkich	Mashg‘ulot sharti
Fan / semestr	Veb dasturlar xavfsizligi / faqat 5-semestr
Shakl	Kompyuter xonasida individual amaliy ish; 30 ta ish joyi
Asosiy vaqt	80 daqiqa; tayyor VM va dastur fayllari bilan
Asosiy natija	JWT sinovlari + BOLA/BFLA tuzatishi + Rate Limiting
Topshiriladi	Hisobot, tuzatilgan app.py, paket versiyalari ro‘yxati
Baholash	10 ball; mezonlar 1.9-bo‘limda

Jihozlar va dasturiy ta’minot

1. Har bir kompyuter: Windows 11, 8 yadroli CPU, RAM 32 GB, SSD 1 TB; lokal tarmoq va internet mavjud.
2. VMware Workstation yoki VirtualBox: Ubuntu 24.04 LTS virtual mashinasi uchun 2 vCPU, RAM 4 GB, disk 25 GB. Windows mezbon tizim bo‘lib qoladi.
3. VM ichida Python 3.12, venv, curl, nano; qo‘shimcha ochiq manbali paketlar: Flask 3.1.3 va PyJWT 2.13.0. O‘rnatish buyruqlari 1.1-bo‘limda.
4. Brauzer yoki curl VMning o‘zida ishlaydi. Server manzili http://127.0.0.1:8008; 30 talaba bir xil portdan o‘z VMida mustaqil foydalanadi.

Tashkiliy shart. 80 daqiqalik reja uchun o‘qituvchi Ubuntu VM, paketlar va ilovalardagi ikki Python faylini oldindan tayyorlaydi. OT o‘rnatish yoki kodni Worddan qo‘lda ko‘chirish zarur bo‘lsa, bu tayyorgarlik alohida vaqt oladi.

Sinov chegarasi. Barcha akkaunt, buyurtma va tokenlar sun’iy. Xizmat faqat VMning loopback manzilida tinglaydi. LANga port yo‘naltirish kerak emas. Amaliy qismda REST ishlatiladi; GraphQL va Gateway qo‘shimcha loyiha tahlilidir.

Qisqa nazariya va kirish siyosati

Autentifikatsiya so‘rov yuboruvchini aniqlaydi; avtorizatsiya esa uning ayni obyekt yoki amal ustidagi huquqini tekshiradi. To‘g‘ri imzolangan token boshqa talabaning buyurtmasini ko‘rish huquqini avtomatik bermaydi. Bu farq A8 variantining asosiy tekshirish nuqtasidir. [1, 2]

Akkaunt	Rol	101-buyurtma	202-buyurtma	Admin summary
ali	student	Ruxsat	Rad	Rad
vali	student	Rad	Ruxsat	Rad
admin	admin	Ruxsat	Ruxsat	Ruxsat

101-buyurtma egasi ali, mahsulot Daftar; 202-buyurtma egasi vali, mahsulot Kitob. Administrator ikkala obyektni ko‘ra oladi. Siyosat koddan oldin belgilanadi: keyingi natijalar shu jadval bilan solishtiriladi.

JWT: mazmunni o‘qish va ishonchni tekshirish

Ushbu HS256 JWT uch qismli: header.payload.signature. Header va payload Base64url ko‘rinishida bo‘lib, shifrlangan emas. Server imzoni, ruxsat etilgan algoritmni, token beruvchisini, auditoriyasini va amal qilish muddatini tekshiradi. sub — foydalanuvchi, iss — beruvchi, aud — qabul qiluvchi API, iat — berilgan vaqt, exp — tugash vaqti. [3]

Laboratoriyada algorithms=['HS256'] serverda qat’iy yozilgan. exp mavjudligi require orqali talab qilinadi; faqat mavjud claimni tekshirish yetarli emas. Rol USERS server jadvalidan olinadi. Shu sababli mijoz yuborgan ixtiyoriy role maydoni huquq manbai emas.

Sinovda qaysi muammo ajratiladi?

5. BOLA — obyekt darajasidagi avtorizatsiya buzilishi: ali tokeni bilan 202-buyurtmani olish. OWASP API1:2023. [1]
6. BFLA — funksiya darajasidagi avtorizatsiya buzilishi: student uchun administrator amalining ochiqligi. OWASP API5:2023. [4]
7. JWTning noto‘g‘ri qabul qilinishi autentifikatsiya bilan bog‘liq; bu variantning umumiy identity() funksiyasi boshidan tekshiruvlarni bajaradi. OWASP API2:2023 bilan bog‘lanadi. [5]
8. So‘rov soni cheklanmasa resurs sarfi nazoratsiz qolishi mumkin. Bu yerda 10 soniyalik sirpanma oynada 5 ta katalog so‘rovi chegarasi ko‘rsatiladi; u barcha DoS holatini bartaraf etmaydi. API4:2023. [6]

Ishonch modeli. issue.py o‘quv token beruvchisining lokal vositasi; login yoki OAuth serveri emas. Talaba tajriba egasi sifatida kalit fayliga ega, lekin TAMPERED sinovida imzo qayta yaratilmaydi. Haqiqiy mijozga jwt.key berilmaydi. HTTP bu yerda faqat loopback uchun; tarmoqdagi Bearer token HTTPS orqali uzatiladi. [2]

1.1. Virtual muhit va dasturni tayyorlash

1-qadam. Tayyor Ubuntu VM bo‘lmasa, VirtualBoxda New yoki VMwareda Create a New Virtual Machine orqali Ubuntu 24.04 LTS ISO faylini ulang. 2 vCPU, 4 GB RAM va 25 GB virtual disk bering. NAT adapter bilan o‘rnating, oddiy foydalanuvchi yarating va terminalni oching. Tayyor VM bo‘lsa shu bosqichni o‘tkazing.

2-qadam. Paketlarni o‘rnating. NAT internetdan paket olish uchun yetadi; bridged adapter yoki inbound port forwarding sozlanmaydi. Quyidagi barcha buyruqlar Ubuntu Bash terminali uchun.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl nano
mkdir -p ~/a8-lab
cd ~/a8-lab
python3 -m venv .venv
source .venv/bin/activate
python -m pip install Flask==3.1.3 PyJWT==2.13.0
python -m pip freeze > requirements-lock.txt
```

3-qadam. 1-ilovadagi A, B va C qismlarni birlashtirib app.py, 2-ilovani issue.py nomida shu katalogga saqlang. nano app.py va nano issue.py orqali tahrirlash mumkin. To‘rtta bo‘sh joyli Python ichki surilishlarini saqlang; sarlavhalar kodga kirmaydi.

```
python -m py_compile app.py issue.py
python issue.py
cp app.py app_before.py
python app.py
```

1-terminalda Running on http://127.0.0.1:8008 ko‘rinadi; uni ochiq qoldiring. issue.py jwt.key va api.key fayllarini faqat ular yo‘q bo‘lsa yaratadi, tokens.env faylini esa yangilaydi. Asosiy tokenlar 2 soat amal qiladi. Kalitni yangilasangiz serverni ham qayta boshlash kerak.

2-terminal: mijozni tayyorlash

```
cd ~/a8-lab
source .venv/bin/activate
source tokens.env
BASE=http://127.0.0.1:8008
curl -i "$BASE/"
curl -i -H "Authorization: Bearer $ALI" "$BASE/api/me"
```

Kutiladigan natija: ikkala javob 200. /api/me javobida sub=ali va role=student. tokens.env Bash o‘zgaruvchilarini belgilaydi; yangi terminalda uni qayta yuklash kerak. Faqat o‘zingiz yaratgan ushbu faylni source qiling.

Dalil D1. VM resurslari, Python/paket versiyalari, serverning 8008 manzili va ali uchun /api/me javobi. Token yoki kalitning to‘liq qiymatini hisobotga qo‘ymang. Token muddati o‘tsa 2-terminalda python issue.py va source tokens.env buyrug‘ini qayta bajaring.

1.2. JWT autentifikatsiyasini tekshirish

1-topshiriq. Quyidagi so‘rovlar bir xil /api/me endpointiga yuboriladi. Faqat credential o‘zgaradi. Token yuborilmaganda server WWW-Authenticate: Bearer sarlavhasi bilan 401 qaytaradi.

```
curl -i "$BASE/api/me"
curl -i -H "Authorization: Bearer $ALI" "$BASE/api/me"
curl -i -H "Authorization: Bearer $TAMPERED" "$BASE/api/me"
curl -i -H "Authorization: Bearer $EXPIRED" "$BASE/api/me"
curl -i -H "Authorization: Bearer $WRONG_AUD" "$BASE/api/me"
curl -i -H "Authorization: Bearer $WRONG_ISS" "$BASE/api/me"
curl -i -H "Authorization: Bearer $NO_EXP" "$BASE/api/me"
```

Sinov tokeni	Nima farq qiladi?	Natija
Yo‘q	Authorization yuborilmagan	401
ALI	Imzo va majburiy claimlar to‘g‘ri	200; ali/student
TAMPERED	sub admin qilib o‘zgargan, eski imzo qolgan	401
EXPIRED	Imzo to‘g‘ri, exp o‘tgan	401
WRONG_AUD	Imzo to‘g‘ri, aud=other-api	401
WRONG_ISS	Imzo to‘g‘ri, iss=other-issuer	401
NO_EXP	Imzo to‘g‘ri, exp maydoni yo‘q	401

Natijani izohlash

issue.py muddati o‘tgan va noto‘g‘ri claimli tokenlarni sinov uchun qonuniy kalit bilan imzolaydi. Bu namunalarda 401ning sababi imzo xatosi emas. TAMPERED namunasida esa payload o‘zgargan va oldingi imzo mos kelmaydi. Shu tariqa tekshiruvlarning har biri alohida kuzatiladi.

```
claims = jwt.decode(
    token, KEY, algorithms=['HS256'],
    issuer='a8-issuer', audience='a8-api',
    options={'require': ['sub', 'iat', 'exp', 'iss', 'aud']})
```

Bu parcha identity()dagi tekshiruv mantiqini ko‘rsatadi; uni alohida bajarish talab qilinmaydi. Algoritm token headeridan olinmaydi. Payloadni oddiy dekodlab ko‘rish imzoni tekshirish o‘rnini bosmaydi. [3]

Dalil D2. Har bir token uchun status jadvali va TAMPERED/EXPIRED farqiga 2–3 gap izoh. Ushbu bosqichda zaif JWT verifier yaratish talab etilmaydi: keyingi zaif endpointlar ham ayni to‘g‘ri identity() tekshiruvidan o‘tadi.

1.3. Obyekt darajasidagi avtorizatsiya — BOLA

2-topshiriq. ali o‘z buyurtmasini olsin, keyin URLdagi IDni 101dan 202ga almashtiring. Ikkinchi so‘rovda yangi token olinmaydi: sub=ali saqlanadi.

```
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/weak/orders/101"
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/weak/orders/202"
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/safe/orders/202"
curl -i -H "Authorization: Bearer $VALI" \
  "$BASE/api/safe/orders/202"
```

Kutiladigan natijalar: 200 va Daftar; 200 va owner=vali, item=Kitob; 403; 200 va Kitob. Zaif endpointda foydalanuvchi aniqlangan, lekin buyurtma egasi bilan solishtirilmagan. Bu BOLA dalilidir. [1]

Ruxsatni to‘liq nazorat qilish

```
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/safe/orders/101"
curl -i -H "Authorization: Bearer $ADMIN" \
  "$BASE/api/safe/orders/202"
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/safe/orders/999"
```

Mos ravishda 200, 200 va 404. Administratorga boshqaning buyurtmasini ko‘rish siyosatda ruxsat etilgan. Mavjud bo‘lmagan 999-ID uchun 404 kelishi boshqa foydalanuvchining mavjud obyektiga qo‘yilgan 403 cheklovidan farq qiladi.

```
def order_safe(user, order):
    if order['owner'] != user['sub'] and user['role'] != 'admin':
        abort(403)
    return order
```

AND sharti muhim: obyekt egasi bo‘lmagan va administrator bo‘lmagan foydalanuvchi rad etiladi. Shartni ORga almashtirish qonuniy egani ham rad etishi mumkin. Tekshiruv obyekt qaytarilishidan oldin bajariladi.

9. IDni UUIDga almashtirish ruxsat tekshiruvining o‘rnini bosmaydi.
10. Haqiqiy bazada odatiy foydalanuvchi uchun so‘rovni id va owner shartlari bilan toraytirish mumkin; administrator yo‘li alohida nazorat qilinadi.
11. Bu mashg‘ulot 403 orqali rad etishni aniq ko‘rsatadi. Obyekt mavjudligini yashirish siyosatida 404 tanlanishi mumkin; tanlov barcha tegishli endpointlarda izchil bo‘lishi kerak.

Dalil D3. Bir xil ali tokeni bilan ID almashganini, zaif javobdagi owner=vali maydonini va himoyalangan 403ni qayd eting. Obyekt egasi va administratorning qonuniy 200 javoblarini ham saqlang.

1.4. Funksiya huquqi va API kaliti

3-topshiriq. Buyurtma egasini tekshirish administrator funksiyalarini avtomatik himoya qilmaydi. Quyidagi endpoint sun’iy A8-ADMIN markerini va buyurtmalar umumiy sonini beradi.

```
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/weak/admin/summary"
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/safe/admin/summary"
curl -i -H "Authorization: Bearer $ADMIN" \
  "$BASE/api/safe/admin/summary"
```

Kutiladigan natijalar: 200 va A8-ADMIN; 403; 200 va orders=2. Birinchi holatda student administrator funksiyasini chaqirdi: bu BFLA. Endpoint nomida /admin mavjudligi o‘zi ruxsat nazorati bo‘la olmaydi. [4]

```
def admin_safe(user):
    if user['role'] != 'admin':
        abort(403)
```

Rol JWT payloadidagi ixtiyoriy role qiymatidan emas, serverdagi USERS jadvalidan olinadi. Ushbu variantda admin funksiyasi faqat o‘qiydi; fayl o‘chirish yoki bazani o‘zgartirish amali mavjud emas.

4-topshiriq. Integratsiya uchun API kaliti

```
curl -i "$BASE/api/integration/ping"
curl -i -H "X-API-Key: $API_KEY" \
  "$BASE/api/integration/ping"
curl -i -H "Authorization: Bearer $ALI" \
  "$BASE/api/integration/ping"
curl -i -H "Authorization: Bearer $API_KEY" "$BASE/api/me"
```

Credential va endpoint	Kutiladigan javob
Kalitsiz integration/ping	401; WWW-Authenticate: ApiKey
To‘g‘ri X-API-Key + integration/ping	200; client=lab-integration
ALI Bearer + integration/ping	401
API kaliti Bearer sifatida + /api/me	401

Bu API kaliti bitta integratsiya mijozini tanitadi; ali yoki vali nomidan ishlamaydi. JWT esa mazkur API uchun foydalanuvchi sub qiymatini beradi. Kalit URL query qismiga yozilmaydi. Ishlab chiqarishda kalit uchun ruxsat doirasi, almashtirish, bekor qilish va alohida kvota kerak. API kaliti yuqori huquqli foydalanuvchi autentifikatsiyasining o‘rnini bosmaydi. [2]

Dalil D4. Student/admin uchun uchta javob va API kaliti bo‘yicha jadval. To‘liq credentialni skrinshotda yashiring; status, endpoint va marker ko‘rinib tursin.

1.5. Rate Limiting va uchta tuzatish

5-topshiriq. Har bir rejimga olti so‘rov yuboring. Bu yuklama sinovi emas: bitta mijozning olti ketma-ket murojaati kifoya. Sikl 10 soniyadan tez yakunlansin; safe katalogiga oldindan so‘rov yubormang yoki avvalgi oynaning tugashini kuting.

```
for mode in weak safe; do
  echo "Rejim: $mode"
  for i in 1 2 3 4 5 6; do
    curl -s -o /dev/null -w "%{http_code}\n" \
      -H "Authorization: Bearer $ALI" "$BASE/api/$mode/catalog"
  done
done
curl -i -H "Authorization: Bearer $ALI" "$BASE/api/safe/catalog"
```

Weak: oltita 200. Safe: beshta 200, keyin 429. Oyna hali tugamagan bo‘lsa oxirgi curl -i ham 429 va Retry-After beradi. Sarlavhadagi soncha soniya kutib, ayni so‘rovni takrorlang: 200 qaytishi kerak. Bu kutishdan oldin VALI tokeni bilan safe katalogni bir marta chaqiring: uning alohida kvotasi sabab 200 bo‘ladi.

limit_safe() monotonic vaqt, sirpanma oyna va Lock bilan ishlaydi. Hisob sub bo‘yicha yuritiladi; yangi token olish o‘sha sub kvotasini yangilamaydi. Limit faqat katalog endpointlariga qo‘llanadi. Xotiradagi holat server qayta boshlanganda yo‘qoladi; ko‘p worker yoki server uchun umumiy hisoblagich kerak. [6]

6-topshiriq. Avvalgi /weak endpointlarini tuzatish

1-terminalda Ctrl+C bosing. app_before.py boshlang‘ich nusxa bo‘lib qolsin. app.py ichidagi quyidagi uchta funksiya tanasini almashtiring:

```
def order_weak(user, order):
    return order_safe(user, order)

def admin_weak(user):
    return admin_safe(user)

def limit_weak(sub):
    return limit_safe(sub)
```

Bu tuzatish himoya funksiyalarini aynan avvalgi marshrutga ulaydi. order_safe va admin_safe ichidagi shartlarni ham tushuntiring; faqat manzilni /safega o‘zgartirish topshiriq bajarilgani hisoblanmaydi.

```
python -m py_compile app.py
python app.py
```

Dalil D5. /weak/orders/202 + ALI endi 403; /weak/admin/summary + ALI ham 403. ALI o‘z buyurtmasiga va ADMIN summaryga 200 olsin. Server qayta boshlanganidan keyin faqat /weak/catalogga oltita so‘rov yuboring: 200 × 5, 429. Tuzatishdan so‘ng /safe va /weak bir sub hisoblagichini bo‘lishadi.

1.6. Vaqt rejasi va qo‘shimcha loyiha tahlili

Bosqich	Vaqt	Natija
Tayyor muhit va nazorat so‘rovi	8 daqiqa	D1
JWT sinovlari	15 daqiqa	D2
BOLA va qonuniy kirishlar	15 daqiqa	D3
BFLA va API kaliti	12 daqiqa	D4
Rate Limiting va uchta tuzatish	20 daqiqa	D5
Hisobot va qisqa himoya	10 daqiqa	D6
Jami	80 daqiqa	Tayyor VM bilan

GraphQL uchun ruxsat qayerda tekshiriladi?

Qo‘shimcha topshiriq. RESTdagi ayni buyurtma xizmatini quyidagi GraphQL so‘roviga moslashtirish rejasini yozing. Bu hujjatdagi Flask kodi /graphql endpointini yaratmaydi; so‘rovni curl bilan mavjud serverga yuborish shart emas.

```
query {
  first: order(id: 101) { id item }
  second: order(id: 202) { id item }
}
```

12. ali uchun first natijasi ruxsatli, second ruxsatsiz bo‘lishi kerak. order resolveri har bir obyekt uchun egani tekshiradi; HTTP so‘rovi bir marta autentifikatsiyadan o‘tishi yetarli emas.
13. Bitta so‘rovdagi ikki alias ikki obyektga murojaat qiladi. Faqat HTTP so‘rovlarini sanash ish hajmini to‘liq ifodalamaydi; query cost, depth va natija soni limitlarini ham rejalashtiring. [7]
14. Hisobotda qisman natija va GraphQL errors maydonini qanday tekshirishingizni yozing. GraphQLda HTTP 200ning o‘zi barcha amallar ruxsat etilganini bildirmaydi.

OAuth/OIDC va API Gateway vazifalari

Loyihada markaziy identifikatsiya xizmati qo‘shilsa, issue.py o‘rnini haqiqiy token beruvchi egallaydi. OAuth 2.0 vakolat berish tizimi, OIDC esa uning ustida autentifikatsiya qatlami bo‘lib xizmat qiladi. API uchun access token ishlatiladi; ID tokenni API credentiali sifatida aralashtirmaslik kerak. Token beruvchisi, auditoriyasi va ruxsatlari tekshiriladi. [8]

Gateway kirish nuqtasida transport, credential tekshiruvi va umumiy kvotani boshqarishi mumkin. Buyurtma kimga tegishli ekanini esa shu obyektni biladigan xizmat tekshiradi. Gateway mavjudligi BOLA tuzatishini bekor qilmaydi. Ushbu variantda Gateway, OAuth yoki OIDC serveri o‘rnatilmagan.

Qo‘shimcha savol. API ikkita workerda ishlasa, hozirgi BUCKETS nega umumiy kvotani kafolatlamaydi? Umumiy ombor va atomar yangilashdan iborat yechimni, GraphQL uchun esa resolver huquqi va query cost nazoratini 5–7 gapda taklif qiling.

1.7. Hisobot shakli va natijalar jadvali

D6 — yakuniy natija jadvali. Quyidagi kutiladigan holatlar yoniga o‘z kuzatuvingiz va dalil raqamini qo‘shing. “Keyin” ustuni tuzatilgan /weak endpointlariga tegishli; JWT tekshiruvi o‘zgarmaydi.

Sinov	Oldin	Keyin	Amaldagi dalil
ALI → /api/me	200	200	_____
JWT: TAMPERED, EXPIRED	401; 401	401; 401	_____
JWT: aud/iss xato; exp yo‘q	401; 401; 401	401; 401; 401	_____
ALI → orders/101	200	200	_____
ALI → orders/202	200; owner=vali	403	_____
VALI → orders/202	200	200	_____
ALI → admin/summary	200; A8-ADMIN	403	_____
ADMIN → admin/summary	200	200	_____
ALI → catalog: 6 so‘rov	200 × 6	200 × 5; 429	_____
VALI → catalog: 1 so‘rov	200	200	_____
Retry-Afterdan keyin ALI	200	200	_____

Hisobot tarkibi

15. Titul: fan, 5-semestr, A8, 1-variant, talaba F.I.Sh., guruh va sana.
16. Maqsad va muhit: VM parametrlari, Python va paket versiyalari, 8008 manzili; uch akkaunt uchun ruxsat siyosati.
17. D1–D5: buyruq, HTTP status va muhim JSON maydoni ko‘ringan skrinshot yoki javob matni; har biriga 1–2 gap izoh.
18. Uchta funksiya uchun oldin/keyin kod; D6 jadvali va 5–7 gap xulosa. API kaliti bo‘yicha to‘rtta natijani alohida qayd eting.
19. Kalit fayllari va tokens.envni hisobotga qo‘shmang. app.py va requirements-lock.txtni ilova qiling; laboratoriya ichida app_before.py taqqoslash uchun saqlanadi.

Tajriba tugagach

Serverni Ctrl+C bilan to‘xtating. Tuzatilgan kod va hisobotni saqlang. Kalitlar va tokenlar individual VMda qoladi; ularni umumiy guruh papkasiga ko‘chirmang. VM snapshotidan foydalanilgan bo‘lsa, hisobotni saqlagandan keyingina o‘qituvchi ko‘rsatmasiga muvofiq holatni tiklang.

Baholashda rad javobi bilan birga qonuniy 200lar ham talab qilinadi. Serverni o‘chirib qo‘yish, barcha so‘rovlarni rad etish yoki 500 xatosini keltirib chiqarish xavfsizlik tuzatishi hisoblanmaydi.

1.8. Muammolarni bartaraf etish

Belgi	Sabab va tekshirish
Connection refused	1-terminalda server ishlayotganini va mijoz ayni VMda ekanini tekshiring.
Address already in use	Takroriy A8 serverini to‘xtating; 8008 portdan bitta jarayon foydalansin.
jwt.key topilmadi	Avval shu katalogda python issue.py bajariladi, keyin python app.py.
No module named jwt	venvni faollashtiring; PyJWT paketini o‘rnating. Boshqa jwt nomli paket kerak emas.
ALI bilan ham 401	source tokens.env bajarilganmi? Token 2 soatdan oshganmi? VM vaqti to‘g‘rimi?
Token yangilangach 401	Kalit fayli almashtirilgan bo‘lsa serverni qayta boshlang; shu loyihaning tokens.env faylini yuklang.
Safe katalogda erta 429	Oldingi murojaatlar shu sub oynasida qolgan. Retry-After muddatidan keyin boshlang.
Oltita so‘rov ham 200	/safe yoki tuzatilgan /weak tanlanganmi? Sikl 10 soniyadan uzoq davom etmadimi?
Tuzatishdan keyin eski holat	app.py saqlansin; server qayta boshlansin. debug=False avtomatik reload qilmaydi.
GraphQL endpoint 404	Bu variantda GraphQL faqat loyiha tahlili; /graphql marshruti o‘rnatilmagan.

1.9. Baholash mezonlari

Mezon	Ball
Muhit, ruxsat siyosati va D1	1
JWT: 7 holat, imzo va claim xatosi farqi	2
BOLA: boshqa obyekt rad etiladi, egasi kira oladi	2
BFLA va API kaliti: rol hamda credential farqi	2
Limit, Retry-After va uchta tuzatishdan keyingi sinov	2
Hisobot, dalillar va nazorat savollariga javob	1
Jami	10

Nazorat savollari

20. Nega JWT imzosi to‘g‘riligi 202-buyurtmaga ruxsat borligini anglatmaydi?
21. expni tekshirish va uning mavjudligini talab qilish qanday farqlanadi?
22. BOLA va BFLA uchun bu laboratoriyadagi qaysi ikkita so‘rov dalil bo‘ldi?
23. Nega API kalitini Bearer JWT o‘rniga qo‘yib bo‘lmaydi?
24. 429dan keyin xizmat qayta ishlashi va VALI kvotasi nega tekshiriladi?
25. GraphQL aliaslari va bir nechta worker holatida qaysi qo‘shimcha nazoratlar kerak?

Manbalar va kod ilovalaridan foydalanish

Mavzu foydalanuvchi taqdim etgan fan dasturidagi A8 bandiga mos. Quyidagi rasmiy manbalar nazariy izohlar va tekshirish tamoyillari uchun ishlatildi. Dastur va sun’iy ma’lumotlar aynan ushbu mashg‘ulot uchun tuzilgan; ular tayyor ishlab chiqarish autentifikatsiya tizimi emas.

[1] OWASP API1:2023 — Broken Object Level Authorization

[2] OWASP — REST Security Cheat Sheet

[3] PyJWT — API Reference: decode, algorithms, require

[4] OWASP API5:2023 — Broken Function Level Authorization

[5] OWASP — API Security Top 10, 2023

[6] OWASP API4:2023 — Unrestricted Resource Consumption

[7] OWASP — GraphQL Cheat Sheet

[8] OpenID Connect Core 1.0 — Authentication va tokenlar

Manbalar tekshirilgan sana: 07.09.2026. O‘quv muhitida sinovdan o‘tgan to‘g‘ridan to‘g‘ri paketlar: Flask 3.1.3, PyJWT 2.13.0. Bog‘liq paketlarning aniq versiyasi requirements-lock.txtda qayd etiladi.

O‘qituvchi uchun tayyorgarlik

26. 1-ilovaning uch qismini app.py, 2-ilovani issue.py qilib bitta katalogga joylang; bo‘sh joylar va oddiy ASCII qo‘shtirnoqlarni saqlang.
27. Kalitsiz boshlang‘ich loyiha va o‘rnatilgan paketli VM nusxasini talabalarga tarqating. Har bir VMda issue.py o‘z kalitini yaratsin; tokenlarni dars boshlanishida chiqaring.
28. VMlarni ommaviy nusxalashdan oldin ishlab turgan A8 serverini to‘xtating. Umumiy nusxaga jwt.key, api.key va tokens.envni qo‘shmang.
29. Asosiy qismda tashqi xizmat, bulut hisobi, pullik API, Postman yoki Burp Suite talab qilinmaydi. Internet paket o‘rnatish va manbalarni o‘qish uchun kerak.
30. Flask ishlab chiqish serveri va xotiradagi USERS/BUCKETS o‘quv soddalashtirishidir. Real xizmatda HTTPS, ishonchli token beruvchi, kalit rotatsiyasi, credentialni bekor qilish va umumiy kvota ombori rejalashtiriladi.

Ilovalarda tokenning o‘zi berilmagan: issue.py uni joyida yaratadi. Bu token muddati sabab keyingi guruhlarda tajriba ishlamay qolishining oldini oladi.

1-ilova. app.py — A qism

A, B va C qismlarni shu tartibda bitta app.py fayliga birlashtiring.

```
import hmac
import math
import time
from collections import defaultdict, deque
from pathlib import Path
from threading import Lock
import jwt
from flask import Flask, abort, jsonify, request
from werkzeug.exceptions import HTTPException

ROOT = Path(__file__).resolve().parent
KEY = (ROOT / 'jwt.key').read_text().strip()
API_KEY = (ROOT / 'api.key').read_text().strip()
ISSUER, AUDIENCE = 'a8-issuer', 'a8-api'
USERS = {'ali': 'student', 'vali': 'student', 'admin': 'admin'}
ORDERS = {
    101: {'id': 101, 'owner': 'ali', 'item': 'Daftar'},
    202: {'id': 202, 'owner': 'vali', 'item': 'Kitob'},
}
app = Flask(__name__)
app.config['MAX_CONTENT_LENGTH'] = 8192
WINDOW, LIMIT = 10, 5
BUCKETS = defaultdict(deque)
LOCK = Lock()

def identity():
    parts = request.headers.get('Authorization', '').split()
    if len(parts) != 2 or parts[0].lower() != 'bearer':
        abort(401)
    try:
        claims = jwt.decode(
            parts[1], KEY, algorithms=['HS256'],
            issuer=ISSUER, audience=AUDIENCE,
            options={'require': ['sub', 'iat', 'exp', 'iss', 'aud']})
    except jwt.InvalidTokenError:
        abort(401)
    sub = claims['sub']
    if not isinstance(sub, str) or sub not in USERS:
        abort(401)
    return {'sub': sub, 'role': USERS[sub]}

def valid_mode(mode):
    if mode not in ('weak', 'safe'):
        abort(404)

def order_weak(user, order):
    return order

def order_safe(user, order):
    if order['owner'] != user['sub'] and user['role'] != 'admin':
        abort(403)
    return order

def admin_weak(user):
    return None

def admin_safe(user):
    if user['role'] != 'admin':
        abort(403)
```

1-ilova. app.py — B qism

A, B va C qismlarni shu tartibda bitta app.py fayliga birlashtiring.

```
def limit_weak(sub):
    return None

def limit_safe(sub):
    now = time.monotonic()
    with LOCK:
        bucket = BUCKETS[sub]
        while bucket and now - bucket[0] >= WINDOW:
            bucket.popleft()
        if len(bucket) >= LIMIT:
            wait = max(1, math.ceil(WINDOW - (now - bucket[0])))
            response = jsonify(error='Too Many Requests')
            response.status_code = 429
            response.headers['Retry-After'] = str(wait)
            return response
        bucket.append(now)
    return None

@app.get('/')
def home():
    return jsonify(lab='A8 / 1-variant', port=8008)

@app.get('/api/me')
def me():
    return jsonify(identity())

@app.get('/api/<mode>/orders/<int:order_id>')
def order_route(mode, order_id):
    valid_mode(mode)
    user = identity()
    order = ORDERS.get(order_id)
    if order is None:
        abort(404)
    func = order_weak if mode == 'weak' else order_safe
    return jsonify(func(user, order))
```

1-ilova. app.py — C qism

A, B va C qismlarni shu tartibda bitta app.py fayliga birlashtiring.

```
@app.get('/api/<mode>/admin/summary')
def admin_route(mode):
    valid_mode(mode)
    user = identity()
    func = admin_weak if mode == 'weak' else admin_safe
    func(user)
    return jsonify(marker='A8-ADMIN', orders=len(ORDERS))

@app.get('/api/<mode>/catalog')
def catalog(mode):
    valid_mode(mode)
    user = identity()
    func = limit_weak if mode == 'weak' else limit_safe
    response = func(user['sub'])
    if response is not None:
        return response
    return jsonify(items=['Daftar', 'Kitob'])

@app.get('/api/integration/ping')
def integration():
    supplied = request.headers.get('X-API-Key', '')
    if not hmac.compare_digest(supplied.encode(), API_KEY.encode()):
        abort(401)
    return jsonify(client='lab-integration', status='ok')

@app.errorhandler(HTTPException)
def errors(error):
    response = jsonify(error=error.name)
    response.status_code = error.code
    if error.code == 401:
        scheme = 'ApiKey' if request.path.startswith(
            '/api/integration/') else 'Bearer'
        response.headers['WWW-Authenticate'] = scheme
    return response

@app.after_request
def response_headers(response):
    response.headers['Cache-Control'] = 'no-store'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    return response

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8008, debug=False)
```

2-ilova. issue.py — lokal o‘quv token beruvchisi

Bu faylni app.py bilan bir katalogga saqlang va serverdan oldin ishga tushiring. Kalitlar faqat yo‘q bo‘lsa yaratiladi; tokens.env har safar yangilanadi.

```
import base64
import json
import secrets
import time
from pathlib import Path
import jwt

root = Path(__file__).resolve().parent
for name in ('jwt.key', 'api.key'):
    path = root / name
    if not path.exists():
        path.write_text(secrets.token_hex(32))
    path.chmod(0o600)
key = (root / 'jwt.key').read_text().strip()
now = int(time.time())
base = dict(iss='a8-issuer', aud='a8-api', iat=now, exp=now + 7200)
tokens = {}
for name in ('ali', 'vali', 'admin'):
    tokens[name.upper()] = jwt.encode(
        dict(base, sub=name), key, algorithm='HS256')
for label, changes in (
    ('EXPIRED', dict(iat=now - 120, exp=now - 60)),
    ('WRONG_AUD', dict(aud='other-api')),
    ('WRONG_ISS', dict(iss='other-issuer')),
):
    claims = dict(base, sub='ali')
    claims.update(changes)
    tokens[label] = jwt.encode(claims, key, algorithm='HS256')
missing = dict(base, sub='ali')
del missing['exp']
tokens['NO_EXP'] = jwt.encode(missing, key, algorithm='HS256')
head, payload, sig = tokens['ALI'].split('.')
forged = json.dumps(dict(base, sub='admin')).encode()
encoded = base64.urlsafe_b64encode(forged).rstrip(b'=').decode()
tokens['TAMPERED'] = '.'.join((head, encoded, sig))
tokens['API_KEY'] = (root / 'api.key').read_text().strip()
target = root / 'tokens.env'
target.write_text(''.join(f"{k}='{v}'\n" for k, v in tokens.items()))
target.chmod(0o600)
print('A8 tokens.env tayyor; amal qilish muddati: 2 soat')
```